EXPERIMENT 2A

Autoencoders and Variational Autoencoders

Comparative implementation of six neural-network models on MNIST

Models covered

No.	Model	Output folder
1	Shallow Autoencoder	shallow_ae
2	Deep Autoencoder	deep_ae
3	Sparse Autoencoder	sparse_ae
4	Variational Autoencoder (VAE)	vae
5	Beta-Variational Autoencoder (β-VAE)	beta_vae
6	Conditional Variational Autoencoder (CVAE)	cvae

Code repository: https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp2a

This report is based on the supplied ZIP archive, including its implementation files, configuration, saved model checkpoints, and reconstruction-image outputs.

1. Aim

To implement and compare six autoencoder-family models on the MNIST handwritten-digit dataset, and inspect how their learned representations reconstruct input images.

2. Objectives

• Implement a shallow autoencoder, deep autoencoder, sparse autoencoder, VAE, β-VAE, and CVAE.
• Train models using a shared configuration and save the best checkpoints.
• Generate reconstruction visualizations for qualitative comparison.
• Evaluate reconstruction behavior using mean squared error (MSE) and pixel reconstruction accuracy.

3. Dataset and Configuration

Setting	Value in supplied config.yaml
Dataset	MNIST
Input dimension	784 (28 × 28 grayscale image flattened)
Latent dimension	16
Number of classes	10
Batch size	128
Training epochs	20
Learning rate	0.001
Weight decay	0.00001
β for β-VAE	4.0
Sparsity coefficient	0.001
Gradient clipping	5.0
Random seed	42

The ZIP's README states that MNIST is downloaded automatically by dataset.py. The archive does not include the dataset itself.

4. Theory

An autoencoder learns to compress an input into a lower-dimensional latent representation and then reconstruct the input from that representation. The encoder maps the input to latent features; the decoder maps those features back to the input space. Reconstruction loss encourages the output to resemble the original image.

A variational autoencoder learns a probabilistic latent representation. Rather than encoding an input to one fixed point, it predicts a latent distribution and samples from it. This supports structured latent spaces and generation, with a reconstruction term and a regularization term in the objective.

5. Model Variants

Shallow Autoencoder: a relatively simple encoder–decoder baseline.
Deep Autoencoder: a deeper network intended to learn more expressive representations.
Sparse Autoencoder: adds a sparsity penalty to encourage only a small number of latent activations to be active.
VAE: learns a probabilistic latent distribution and balances reconstruction with latent-distribution regularization.

β**-VAE:** uses a β-weighted regularization term; the supplied configuration sets β = 4.0.
CVAE: conditions the model on the digit label, allowing the label to influence reconstruction/generation.

6. Procedure

1. Install the dependencies listed in requirements.txt.
2. Load the MNIST dataset using dataset.py.
3. Read configuration values from config.yaml.
4. Build the selected architecture through models.py.
5. Train a model using train.py; the supplied README also provides run_all.py to run all six variants.
6. Save the best checkpoint and reconstruction image under the corresponding outputs/<model_name>/ directory.
7. Run evaluate.py with a model name and checkpoint to calculate pixel reconstruction accuracy and MSE.

Commands documented in the supplied README

```
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
```

Train all six models:

```
python run_all.py
```

Train one model:

```
python train.py --model deep_ae
```

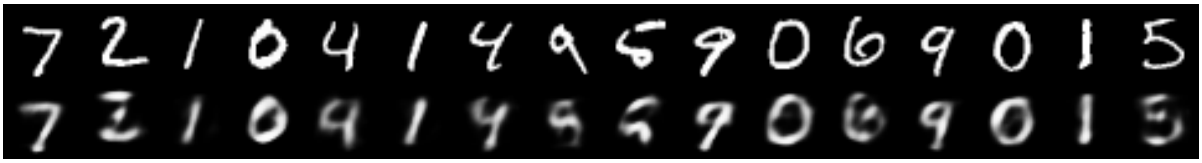
Evaluate a saved checkpoint:

```
python evaluate.py --model deep_ae --checkpoint outputs/deep_ae/best.pt
```

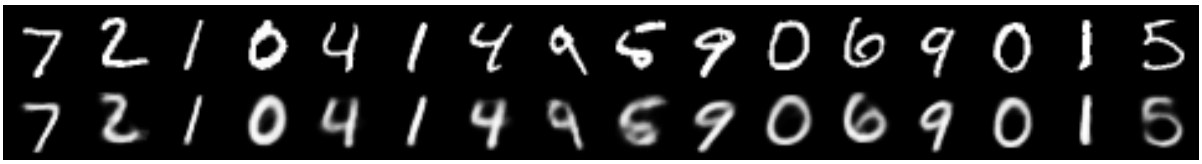
7. Results and Observations

The ZIP contains saved checkpoints for all six variants and one reconstruction visualization per model. The figures below reproduce the supplied output images; the upper row shows the input digits and the lower row shows the corresponding reconstructions.

Shallow Autoencoder



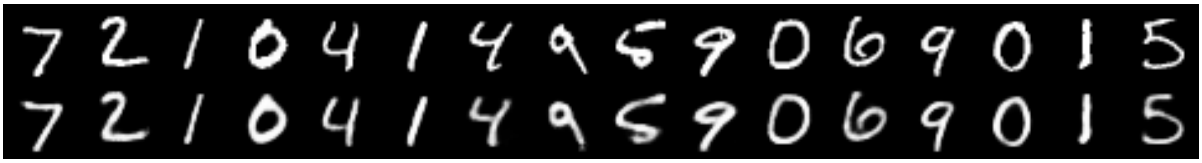
Deep Autoencoder



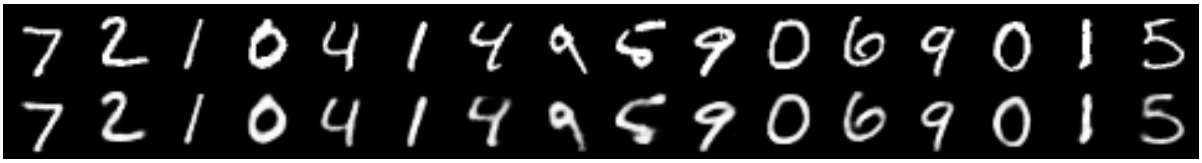
Sparse Autoencoder



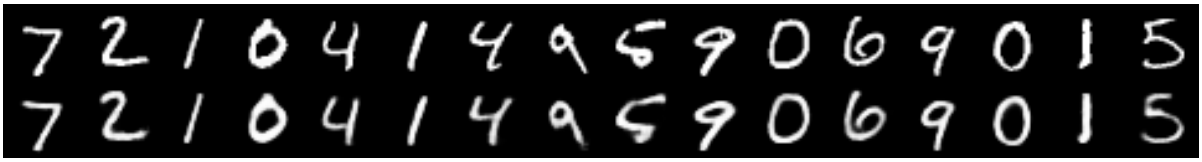
Variational Autoencoder (VAE)



Beta-VAE



Conditional VAE (CVAE)



Qualitative observations

• All six saved visualizations show reconstructed digit patterns that broadly resemble the corresponding inputs.
• The shallow autoencoder's reconstructions appear somewhat less crisp in places than those from the deeper/sparse variants.
• The deep and sparse autoencoder visualizations preserve the overall digit shapes clearly in the examples shown.
• The VAE, β-VAE, and CVAE outputs are also recognizable, with some smoothing or small differences from the inputs.
• These are visual observations from the provided sample images, not a numerical ranking of all test examples.

Numerical results limitation

The archive contains evaluate.py, which calculates pixel reconstruction accuracy and MSE, but it does not contain captured metric logs or a results table. Therefore, no numerical scores are invented in this report. To obtain exact metrics, run the documented evaluation command for each checkpoint on the MNIST test split.

8. Evaluation Metrics

Mean Squared Error (MSE): measures the average squared difference between reconstructed pixel values and original pixel values. Lower MSE indicates closer numerical reconstruction.

Pixel reconstruction accuracy: the supplied implementation thresholds the reconstruction and input pixels and measures the fraction that agree. The project README notes that reconstruction loss and MSE should be considered together with pixel accuracy.

Example evaluation commands

```
python evaluate.py --model shallow_ae --checkpoint outputs/shallow_ae/best.pt

python evaluate.py --model deep_ae --checkpoint outputs/deep_ae/best.pt

python evaluate.py --model sparse_ae --checkpoint outputs/sparse_ae/best.pt

python evaluate.py --model vae --checkpoint outputs/vae/best.pt

python evaluate.py --model beta_vae --checkpoint outputs/beta_vae/best.pt

python evaluate.py --model cvae --checkpoint outputs/cvae/best.pt
```

9. Conclusion

The supplied implementation covers six autoencoder-family approaches on MNIST and saves both trained checkpoints and reconstruction examples. The available figures show that each model learned to reproduce recognizable handwritten-digit structure. The model families differ in their objectives: the basic and deep autoencoders focus on reconstruction, the sparse model encourages sparse latent activity, and the VAE-family models learn probabilistic latent representations, with β-VAE changing the strength of regularization and CVAE using class labels as conditions. A reliable quantitative comparison requires running evaluate.py and recording MSE and pixel reconstruction accuracy for every model.

10. Project Files

models.py — defines model architectures and model construction.
dataset.py — prepares MNIST data loaders.
train.py — training procedure for a selected model.
run_all.py — runs the model set in sequence.
evaluate.py — evaluates a checkpoint using MSE and pixel reconstruction accuracy.
utils.py — utility functions, including device selection and pixel accuracy.
config.yaml — experiment configuration.
outputs/ — saved checkpoints and reconstruction figures.

Code link

https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp2a

Reference note

The uploaded one-page manual is titled “Experiment 9” and describes LIME-based sentiment explanation using a DistilBERT sentiment model. Its subject does not match the Autoencoders/VAE code and outputs in the supplied ZIP. This report therefore uses the ZIP as the evidence for Experiment 2A and does not attribute the LIME procedure to this experiment.